

2 Marks:

1. State any two Operation of Bitcoin Blockchain.
2. State the four role of blockchain oracles in smart contract execution
3. Identify the main components of a blockchain block
4. Define a block in blockchain technology
5. Define P2P networks in blockchain systems with one example
6. Identify any two design primitives of blockchain
7. Identify the consensus mechanism in blockchain.
8. Mention four advantage of Proof of Work consensus.).
9. List any two examples of P2P applications with explain.
10. Classify the concept of Byzantine Fault Tolerance(BFT).
11. State four componets of Ethereum
12. Identify 4 application of smart contracts
13. Distinguish between Bitcoin scripting and Ethereum smart contracts.
14. State the four role of blockchain oracles in smart contract execution
15. Define Wallets for Ethereum.
16. Identify 4 componens of contracts.
17. Classify the concept of Decentralized Applications (DApps)
18. List out four Decentralized Applications (DApps)

16 Marks:**1. Briefly explain the concept of Blockchain Architecture with neat diagram. (unit 3)**

Blockchain architecture refers to the structural design and functional arrangement of components that enable a blockchain network to operate in a decentralized, secure, and transparent manner. It defines how data is created, validated, stored, linked, and shared across distributed nodes without the involvement of a central authority. The architecture integrates cryptographic techniques, peer-to-peer networking, consensus mechanisms, and layered system design to ensure immutability and trust.

COMPONENTS OF BLOCKCHAIN**Node**

A node is a computer system that participates in the blockchain network and maintains a copy of the distributed ledger. Unlike traditional centralized systems where a single server stores data, blockchain relies on multiple nodes spread across different geographical locations. Each node independently verifies transactions and blocks before accepting them into its copy of the ledger. Nodes can be categorized into full nodes, which store the complete blockchain and validate every transaction, and light nodes, which store partial data and rely on full nodes for verification. Some nodes also function as mining nodes, performing complex computational tasks to create new blocks. Because thousands of nodes maintain identical copies of the ledger, blockchain achieves decentralization, transparency, and fault tolerance.

Block

A block is a data structure that stores a set of validated transactions. It acts as a container that permanently records transaction details. Every block consists of two main sections: the block header and the block body. The body contains the list of transactions, while the header contains metadata required to identify and secure the block.

Block Header

The block header is the identity section of the block. It contains essential metadata that uniquely represents the block in the blockchain network. The header plays a critical role in linking blocks and ensuring security.

Hash

A hash is a fixed-length output generated from input data using a cryptographic hash function such as SHA-256. It acts as a digital fingerprint of the block. Even a minor change in the block's data results in a completely different hash value. This property, known as the avalanche effect, ensures data integrity. Hashing makes it

practically impossible to alter data without detection, thereby securing the blockchain. In networks such as Bitcoin, SHA-256 hashing is used to generate block hashes and secure transactions.

Merkle Tree

A Merkle Tree is a hierarchical data structure used to efficiently organize and verify transactions within a block. Each transaction is first hashed individually. These hashes are then paired and hashed again repeatedly until only one final hash remains. This final hash is called the Merkle Root and is stored in the block header. The Merkle Tree ensures that if even a single transaction changes, the Merkle Root also changes, indicating tampering. It allows efficient verification of transactions without requiring access to the entire dataset, improving scalability and performance.

Timestamp

The timestamp records the exact date and time at which the block is created. It ensures that blocks are arranged in chronological order. By maintaining time records, blockchain prevents double spending and maintains transaction sequence integrity. Timestamping also provides historical traceability of all activities in the ledger.

Previous Hash

The previous hash is the hash value of the preceding block in the chain. It establishes a connection between consecutive blocks. This linking mechanism forms the “chain” in blockchain. If any block is altered, its hash changes, which breaks the link with the subsequent block. Therefore, modifying one block requires modifying all subsequent blocks, which is computationally infeasible. This property ensures immutability.

Nonce

The nonce, meaning “number used once,” is a variable field in the block header used during mining. Miners repeatedly change the nonce value to generate different hash outputs until a hash satisfying the network’s difficulty target is found. The correct nonce proves that sufficient computational work has been performed, which is central to the Proof of Work consensus mechanism.

Transaction

A transaction represents the transfer of digital assets, information, or value between participants in the blockchain network. Every transaction is digitally signed using cryptographic keys to ensure authenticity. Once created, the transaction is broadcast to the network, where nodes verify its validity before including it in a block. After confirmation and addition to a block, the transaction becomes permanent and irreversible. For eg., in Ethereum, transactions may represent cryptocurrency transfers or execution of smart contract functions.

Chain

The chain refers to the continuous linking of blocks using previous hash references. The first block in the blockchain is known as the Genesis Block. Each new block strengthens the overall security of the system. Because every block depends on its predecessor, altering one block disrupts the entire chain. This interconnected structure provides immutability and transparency.

Miners

Miners are specialized nodes responsible for validating transactions and creating new blocks. They collect pending transactions, organize them into a candidate block, and compete to solve complex mathematical puzzles. In Proof of Work systems, miners attempt to find a valid nonce that produces a hash below the target difficulty level. Once a valid solution is found, the block is broadcast to the network for verification. Miners are incentivized through rewards, which may include newly generated cryptocurrency and transaction fees.

Consensus

Consensus is the mechanism through which all nodes in the network agree on the validity of transactions and blocks. Since blockchain operates without a central authority, consensus algorithms ensure that all participants maintain a consistent ledger. Common consensus mechanisms include Proof of Work (PoW) and Proof of Stake (PoS). In PoW, computational effort determines block validation, whereas in PoS, validators are selected based on the amount of cryptocurrency they stake. Consensus ensures trust, prevents double spending, and maintains synchronization across distributed nodes.

NEAT DIAGRAM OF BLOCKCHAIN

BLOCK 1	BLOCK 2	BLOCK 3
-----	-----	-----
Previous Hash: 0	-> Previous Hash: H1	-> Previous Hash: H2
Merkle Root	Merkle Root	Merkle Root
Timestamp	Timestamp	Timestamp
Nonce	Nonce	Nonce
Block Hash (H1)	Block Hash (H2)	Block Hash (H3)
-----	-----	-----
Transactions	Transactions	Transactions

ARCHITECTURE OF BLOCKCHAIN (5 LAYERS)

Blockchain architecture can also be understood through a layered model consisting of five layers.

1. Hardware Layer

The hardware layer forms the physical foundation of the blockchain network. It includes servers, storage devices, networking infrastructure, and computational systems. Nodes operate on physical machines or cloud-based infrastructure. Virtual Machines (VMs) provide abstraction of computing resources, while containers such as Docker allow lightweight deployment of blockchain services. This layer ensures distributed storage and computational power necessary for mining, validation, and transaction processing.

2. Data Layer

The data layer defines how information is structured and secured within the blockchain. It includes cryptographic hash functions, digital signatures, Merkle Trees, and block data structures. Digital signatures authenticate users and prevent unauthorized access. Hashing ensures immutability, while Merkle Trees organize transaction data efficiently. This layer guarantees integrity, security, and tamper resistance of stored information.

3. Network Layer

The network layer operates on a peer-to-peer (P2P) communication model. Unlike traditional client-server systems, blockchain nodes communicate directly with each other. Transactions initiated by users are broadcast to all nodes. Newly created blocks are propagated throughout the network for validation. The P2P structure enhances decentralization, resilience, and transparency.

4. Consensus Layer

The consensus layer ensures agreement among distributed nodes regarding the state of the blockchain. It validates transactions and determines which node has the authority to add the next block. This layer prevents malicious activities and ensures ledger consistency. By implementing algorithms such as Proof of Work and Proof of Stake, it maintains synchronization and trust without centralized control.

5. Application Layer

The application layer is the topmost layer where end-users interact with the blockchain. It includes smart contracts, decentralized applications (DApps), and cryptocurrency platforms. Smart contracts are self-executing programs that automatically enforce predefined rules. DApps operate without central servers and use blockchain for backend functionality. Industries such as finance, healthcare, supply chain management, and voting systems use blockchain applications at this layer. For instance, Ethereum provides a robust environment for developing smart contracts and decentralized applications.

-
2. Illustrate the Consensus protocols for Permissioned Block chains with real time examples(unit3)
 3. Explain how cryptocurrencies and Bitcoin work and how they are used in real-world applications. (unit3)

Cryptocurrencies represent a revolutionary form of digital money that operates without the control of any central authority such as a bank or government. They are built on blockchain technology, which provides a decentralized, transparent, and tamper-proof ledger system. The core idea behind cryptocurrencies is to enable secure peer-to-peer transfer of value over the internet without relying on trusted intermediaries.

Before cryptocurrencies, digital payments always required a third party such as a bank to verify balances, authorize transactions, and maintain records. Cryptocurrencies replace this trust in institutions with trust in mathematics, cryptography, and distributed consensus. The first successful implementation of this idea was Bitcoin, introduced in 2009 by Satoshi Nakamoto.

What is a Cryptocurrency?

A cryptocurrency is a digital or virtual currency that uses cryptographic techniques to secure transactions, verify ownership, and control the creation of new units. It exists entirely in digital form and is recorded on a public distributed ledger known as the blockchain. The defining characteristics of cryptocurrencies are decentralization, transparency, immutability, and security. They are generated and maintained by network participants following predefined protocols.

Bitcoin: The First Cryptocurrency

Bitcoin is the first decentralized cryptocurrency and remains the most influential example of blockchain-based digital money. It was designed as an alternative to traditional financial systems, especially after the 2008 financial crisis, where trust in banks declined. Bitcoin allows users to send money directly to each other without banks, payment gateways, or intermediaries. Every transaction is recorded on a public blockchain that anyone can verify, making the system transparent and resistant to fraud.

Core Components Required for Bitcoin to Work

To understand how Bitcoin works, it is necessary to understand wallets, cryptographic keys, transactions, blockchain storage, mining, and consensus.

Wallets and Cryptographic Keys

A Bitcoin wallet is a software or hardware application that manages cryptographic keys. It does not store actual bitcoins but stores the private and public keys that prove ownership of bitcoins. Each user has a private key, which must be kept secret, and a public key, which is shared with others. A wallet address is generated from the public key and is used to receive Bitcoin. Ownership of Bitcoin is mathematically proven through possession of the private key.

Detailed Working of Bitcoin with Example

“A Pays B 2 BTC”

To clearly understand how Bitcoin works, consider a practical transaction.

Assume that Person A owns 5 BTC and Person B owns 1 BTC. Person A wants to send 2 BTC to Person B.

Transaction Creation

Person A opens their Bitcoin wallet and enters Person B's wallet address and the amount of 2 BTC. The wallet software constructs a transaction using Bitcoin's UTXO (Unspent Transaction Output) model. This means A's wallet selects previous unspent outputs worth 5 BTC as input and creates two outputs: one output of 2 BTC to B and another output of approximately 3 BTC back to A as change, minus a small transaction fee.

Digital Signature and Authentication

The transaction is digitally signed using A's private key. This digital signature proves that A is the legitimate owner of the Bitcoin being spent and has authorized the transaction. The private key is never shared with the network. Instead, nodes use A's public key to verify the authenticity of the signature.

Broadcasting and Network Verification

Once signed, the transaction is broadcast to the Bitcoin peer-to-peer network. Thousands of nodes receive the transaction and independently verify it. They check whether A has sufficient balance, whether the signature is valid, and whether the Bitcoin has not already been spent.

After verification, the transaction is placed in a pool of unconfirmed transactions called the mempool.

Mining and Proof of Work

Miners collect transactions from the mempool and bundle them into a block. To add this block to the blockchain, miners must solve a cryptographic puzzle known as Proof of Work. This involves finding a nonce such that the hash of the block header is below a predefined difficulty target. This process requires massive computational effort, making it extremely expensive to manipulate the blockchain. The first miner to find a valid solution broadcasts the block to the network.

Block Confirmation and Final Settlement

Other nodes verify the block's validity and the correctness of the Proof of Work. Once accepted, the block is added to the blockchain. At this point, the transaction from A to B is confirmed. B's wallet now shows an additional 2 BTC, and A's balance reduces accordingly.

After several additional blocks are added on top of this block, the transaction becomes practically irreversible.

Neat Diagram: Bitcoin Transaction Flow

A's Wallet

(Private Key)

|

| 1. Create & Sign Transaction (2 BTC)

v

Peer-to-Peer Network

|

| 2. Verification by Nodes

v

Mempool (Unconfirmed Transactions)

|

| 3. Mining (Proof of Work)

v

New Block Added to Blockchain

|

| 4. Confirmation

v

B Receives 2 BTC

How Bitcoin Prevents Double Spending

Bitcoin solves the double-spending problem using a public blockchain ledger and Proof of Work consensus. Since all transactions are publicly recorded and verified by the majority of the network, a user cannot spend the same Bitcoin twice. To reverse a transaction, an attacker would need to control more than 51% of the network's computational power, which is practically infeasible in a large decentralized network.

Real-World Applications of Bitcoin and Cryptocurrencies

- **Peer-to-Peer Payments:** Bitcoin enables direct peer-to-peer payments without intermediaries. Individuals can send money globally within minutes, regardless of geographical location.
- **Cross-Border Remittances:** Bitcoin significantly reduces cost and time in international money transfers. Migrant workers can send money to families without relying on expensive remittance services.

- **Investment and Store of Value:** Bitcoin is often referred to as “digital gold” due to its limited supply of 21 million coins. Investors use it as a hedge against inflation and currency devaluation.
 - **E-Commerce and Online Payments:** Some merchants accept Bitcoin for online purchases, reducing transaction fees and eliminating chargeback fraud.
 - **Financial Inclusion:** Cryptocurrencies allow unbanked populations to participate in the global financial system using only a smartphone and internet connection.
 - **Smart Contracts and Decentralized Applications:** While Bitcoin focuses on payments, platforms such as Ethereum extend the concept by supporting smart contracts and decentralized applications, enabling use cases in finance, insurance, supply chain, and governance.
-

4. Evaluate the suitability of Elliptic Curve Cryptography for resource-constrained environments with example.

5. Explain the working of smart contracts with a neat diagram (unit 4)

A Smart Contract is a self-executing digital agreement stored on a blockchain that automatically enforces, verifies, and executes the terms of a contract when predefined conditions are satisfied, without the need for intermediaries. Smart contracts are most widely implemented on platforms like Ethereum, where they run as decentralized programs on the blockchain network.

Working of Smart Contracts (20 Marks – Detailed and Expanded Explanation)

The working of a smart contract can be explained in a structured sequence of stages. Each stage plays an important role in ensuring automation, security, and decentralization.

1. Requirement Analysis and Contract Design

Before coding, the contract terms must be clearly defined. This includes:

1. Identifying participating parties
2. Defining assets involved (money, property, tokens)
3. Specifying conditions for execution
4. Determining penalties or fallback actions
5. Setting deadlines or time constraints

The logic must be precise because blockchain execution is irreversible. Example condition format:

- IF X happens → THEN perform Y
- IF deadline not met → THEN refund

This logical structuring is the foundation of smart contract functioning.

2. Coding the Smart Contract

After defining the agreement:

- The contract is written in a blockchain programming language.
- On Ethereum, it is usually written in Solidity.
- The code contains:
 - Functions
 - Variables
 - Access modifiers
 - Conditional statements
 - Payment logic

Example (conceptual logic):

If payment $\geq$ required amount

Then transfer ownership

The contract converts legal terms into executable code. Once coded incorrectly, errors may lead to financial loss.

3. Compilation and Deployment

After coding:

1. The smart contract is compiled into bytecode.
2. A deployment transaction is sent to the blockchain.
3. Miners/validators validate the transaction.
4. After confirmation, the contract is permanently stored.
5. The contract gets a unique blockchain address.

Deployment ensures:

- Immutability
- Transparency
- Decentralized storage

At this stage, the contract becomes active and accessible.

4. Broadcasting and Network Validation

Whenever a user interacts with the contract:

- A transaction request is generated.
- The request is digitally signed.
- It is broadcast to the blockchain network.
- Nodes verify:
 - Sender authenticity
 - Digital signature validity
 - Sufficient balance
 - Compliance with contract rules

Only valid transactions are accepted. This stage prevents fraud and unauthorized access.

5. Execution of Smart Contract Logic

Once validated:

- The contract executes automatically.
- It checks predefined conditions.
- Based on logic, it performs actions.

Execution follows deterministic rules: Every node executing the contract produces the same output.

Possible actions include:

- Transfer cryptocurrency
- Update ownership records
- Release digital assets
- Trigger another contract

No human intervention is required.

6. Consensus Mechanism

Before final confirmation:

- The network must agree on the transaction result.
- Consensus mechanisms (Proof of Work / Proof of Stake) validate the block.
- Once consensus is achieved, the transaction becomes permanent.

This ensures:

- No double spending
 - No data manipulation
 - Network-wide agreement
-

7. Updating Blockchain State

After execution:

- Account balances change.
- Contract variables update.
- Ownership details modify.

- Transaction record is stored permanently.

This updated state is replicated across all nodes. Immutability ensures that once recorded, the data cannot be altered.

8. Event Logging and Transparency

Smart contracts generate event logs. These logs:

- Record actions performed.
- Allow auditing.
- Increase transparency.
- Enable DApps to track results.

Every action is publicly verifiable on the blockchain explorer.

9. Interaction Through Decentralized Applications (DApps)

Users generally do not interact directly with contract code. Instead:

User → DApp Interface → Smart Contract → Blockchain

The DApp provides:

- User-friendly interface
- Wallet integration
- Transaction confirmation prompts

The backend logic remains decentralized.

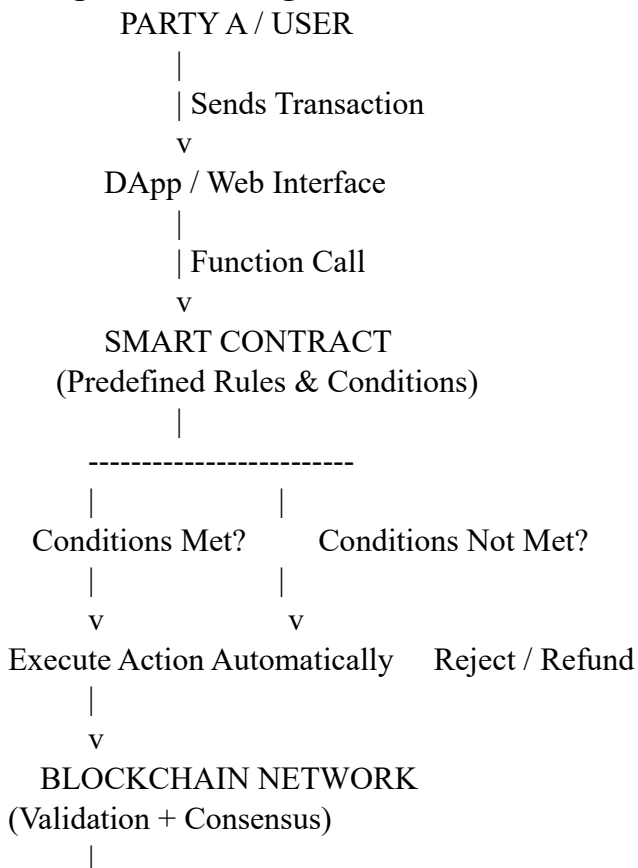
10. Completion and Enforcement

Once conditions are fulfilled:

- The contract enforces outcomes automatically.
- Funds are released.
- Ownership transfers.
- Agreement closes.

Unlike traditional contracts, no court or third-party enforcement is required. The code itself enforces compliance.

Neat Diagram – Working of Smart Contract



Example – Smart Contract in Insurance

Traditional Insurance Process:

1. Customer files claim
2. Insurance company verifies
3. Manual approval
4. Payment release

Smart Contract Insurance Model:

1. Weather data feeds into contract
2. If rainfall < threshold
3. Contract automatically releases compensation

No paperwork. No manual approval.

Characteristics Observed in Working

From the entire process, smart contracts demonstrate:

1. Automation – Executes automatically when triggered
 2. Deterministic Output – Same result across all nodes
 3. Immutability – Cannot be altered after deployment
 4. Transparency – Publicly verifiable
 5. Trustlessness – No need to trust other party
 6. Security – Cryptographic validation
-

Advantages of Smart Contract Working

1. Eliminates intermediaries
 2. Reduces operational costs
 3. Speeds up transaction processing
 4. Minimizes human error
 5. Prevents fraud and manipulation
 6. Ensures accuracy of execution
 7. Provides auditability
-

Limitations of Smart Contract Working

1. Coding errors cannot be easily fixed
 2. Legal enforceability varies across countries
 3. Scalability issues in high-traffic networks
 4. Requires technical knowledge
 5. Irreversible once executed
-

Comparison: Traditional Contract vs Smart Contract

Feature	Traditional Contract	Smart Contract
Enforcement	Legal system	Code execution
Speed	Slow	Instant
Intermediaries	Required	Not required
Transparency	Limited	High
Modification	Possible	Difficult

Overall Summary of Working

The working of a smart contract follows a structured lifecycle:

1. Agreement terms defined
2. Code written
3. Contract deployed
4. User triggers transaction
5. Network validates
6. Conditions checked
7. Automatic execution
8. State updated
9. Record permanently stored

Thus, smart contracts operate as autonomous digital agreements running on blockchain platforms like Ethereum, ensuring secure, transparent, decentralized, and automated execution of contractual obligations.

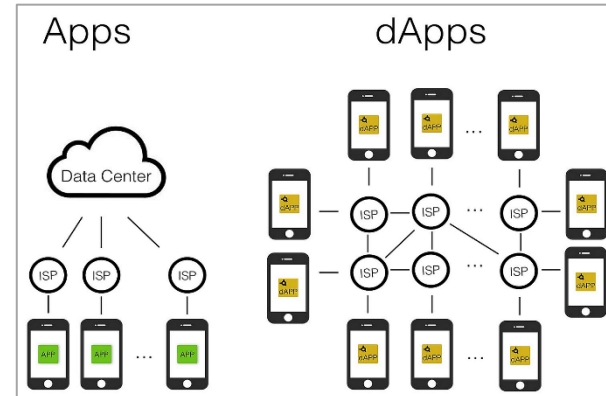
-
6. Compare Bitcoin scripting and Ethereum smart contracts with respect to real-world applications (unit 4)
 7. Briefly explain the concept of Ethereum Virtual Machine (EVM) with neat diagram. (unit4)
 8. Illustrate the concept of decentralized and explain the real-world applications.

Decentralized Applications, commonly known as DApps, represent a major evolution in software architecture where applications operate on decentralized blockchain networks instead of centralized servers. Traditional applications such as social media platforms, banking apps, or e-commerce websites rely on centralized servers owned and controlled by a single organization. In contrast, DApps function on distributed blockchain networks where control is shared among participants rather than concentrated in one authority. The idea behind DApps is to remove intermediaries, increase transparency, enhance security, and give users control over their data and digital assets. Most DApps today are built on blockchain platforms such as Ethereum, which support smart contracts — self-executing programs that run automatically when predefined conditions are met.

Concept of Decentralized Applications (DApps)

A Decentralized Application is a software application that runs on a blockchain or peer-to-peer network rather than on a centralized server. The backend logic of a DApp is executed by smart contracts deployed on a blockchain, while the frontend may appear similar to traditional web or mobile applications.

The fundamental characteristics of a DApp include decentralization, transparency, immutability, and autonomy. Unlike centralized apps where data is stored in a single database, DApps store data across distributed nodes in a blockchain network. This ensures that no single entity has full control over the application. In centralized systems, users must trust the organization managing the application. In DApps, trust is replaced with code. Smart contracts automatically enforce rules without requiring human intervention.



Architecture of a Decentralized Application

A DApp typically consists of three main components:

1. Frontend Interface

The frontend is the user interface through which users interact with the application. It may be developed using standard web technologies such as HTML, CSS, and JavaScript. From a user's perspective, it may look similar to any regular website or mobile application. However, instead of communicating with a centralized server, the frontend interacts with the blockchain network using special libraries or wallet software.

2. Smart Contracts (Backend Logic)

The backend logic of a DApp is implemented using smart contracts. A smart contract is a self-executing program stored on the blockchain that automatically performs actions when specific conditions are satisfied.

For example, in a decentralized lending DApp, a smart contract may automatically release funds when collateral conditions are met. Since smart contracts run on blockchain, their execution is transparent and cannot be altered once deployed.

Smart contracts remove the need for intermediaries because rules are enforced automatically through code.

3. Blockchain Network

The blockchain acts as the distributed database that stores all transactions and smart contract states. Every action taken within the DApp is recorded on the blockchain, making it transparent and immutable. Each node in the blockchain network maintains a copy of the ledger, ensuring decentralization and fault tolerance.

Illustration: Working of a DApp

Below is a simplified conceptual diagram showing how a DApp operates:

User

|

| 1. Interaction (Request)

v

Frontend Interface (Web/App)

|

| 2. Calls Smart Contract

v

Smart Contract (Blockchain)

|

| 3. Validation & Execution

v

Blockchain Network (Distributed Ledger)

|

| 4. Result Returned

v

User Receives Output

Explanation of flow:

1. The user interacts with the DApp through the frontend interface.
 2. The frontend sends a request to the smart contract.
 3. The smart contract executes logic on the blockchain.
 4. The result is recorded permanently and displayed to the user.
-

Features of Decentralized Applications

Decentralization ensures no single authority controls the application.

Transparency allows anyone to verify transactions.

Immutability ensures that data once recorded cannot be changed.

Autonomy allows smart contracts to operate without human interference.

Security is strengthened because data is distributed across multiple nodes.

Real-World Applications of DApps

Decentralized applications are being used across multiple industries. The following sections explain practical real-world use cases.

1. Decentralized Finance (DeFi)

One of the most significant applications of DApps is in the financial sector, known as Decentralized Finance (DeFi). DeFi platforms allow users to lend, borrow, trade, and earn interest without banks. For e.g., instead of depositing money in a traditional bank to earn interest, users can deposit cryptocurrency into a DeFi DApp. Smart contracts automatically manage lending, borrowing, and interest distribution. Advantages include:

- No intermediaries
- Lower fees
- Global accessibility
- Transparent transactions

This system eliminates traditional banking restrictions and allows anyone with internet access to participate.

2. Decentralized Exchanges (DEX)

Traditional cryptocurrency exchanges are centralized and can be hacked or manipulated. Decentralized Exchanges operate as DApps where users trade cryptocurrencies directly from their wallets without transferring funds to a central authority. Smart contracts handle trade execution automatically. This reduces risks such as exchange hacks, fraud, and withdrawal restrictions.

3. Supply Chain Management

DApps improve transparency in supply chains by recording every step of a product's journey on blockchain. For example, in the food industry:

- A product's origin is recorded.
- Transportation details are logged.
- Storage conditions are verified.

Consumers can trace the product history, ensuring authenticity and preventing fraud.

4. Voting Systems

Traditional voting systems face issues such as tampering, fraud, and lack of transparency. A blockchain-based voting DApp allows:

- Secure vote casting
- Transparent vote counting
- Immutable records

Each vote is recorded on the blockchain and cannot be altered. This enhances election integrity and trust.

5. Gaming Industry

Blockchain-based gaming DApps allow players to own in-game assets such as skins, weapons, or characters as digital tokens. Unlike traditional games where developers control assets, players have full ownership and can trade assets securely. This introduces a play-to-earn model, where users can earn cryptocurrency through gameplay.

6. Social Media Platforms

Traditional social media platforms control user data and monetize content without fair revenue sharing.

Decentralized social media DApps:

- Give users control over data.
 - Reduce censorship.
 - Allow creators to monetize content directly through cryptocurrency.
 - Content ownership remains with users rather than corporations.
-

7. Healthcare Systems

DApps can store patient records securely on blockchain. Access can be granted through cryptographic keys, ensuring privacy and preventing unauthorized modifications. Patients gain control over their medical history and can share records securely with doctors or hospitals.

8. Real Estate and Property Management

Smart contracts can automate property transactions. For example:

- Buyer sends payment.
- Smart contract verifies payment.
- Ownership records are automatically updated.

This eliminates middlemen such as brokers and reduces paperwork.

Advantages of DApps

DApps provide higher security due to decentralization.

They reduce operational costs by removing intermediaries.

They increase transparency and trust.

They prevent single-point failures.

They provide global accessibility.

Limitations of DApps

Despite advantages, DApps face challenges such as scalability issues, slower transaction speeds compared to centralized apps, higher transaction costs during network congestion, and complex UIs. Additionally, once a smart contract is deployed, modifying it is difficult, which requires careful coding and auditing.

Comparison: Centralized Apps vs Decentralized Apps

Centralized applications rely on company-owned servers and dbs, whereas DApps rely on blockchain networks. In centralized systems, users must trust the organization managing data. In DApps, trust is placed in transparent code and consensus mechanisms. Centralized apps can modify or delete user data, while DApps maintain immutable records. However, centralized systems generally offer faster performance and easier updates.

MODEL UNIT 5

2 marks:

1. List any two security challenges in blockchain

Security challenges refer to vulnerabilities or risks that can compromise the confidentiality, integrity, and availability of blockchain systems such as Bitcoin.

1. **Scalability Issues** As the number of users and transactions increases, the network becomes slower and more difficult to manage securely.
 2. **Private Key Management** Loss or theft of private keys can lead to permanent loss of assets, as there is no central authority to recover them.
-

2. State the four main objectives of Selfish Mining

Selfish mining is a strategy where a miner (or group) deliberately withholds mined blocks to gain an unfair advantage over honest miners.

1. **Maximize Mining Rewards** The attacker aims to earn more rewards than their fair share.
 2. **Waste Honest Miners' Effort** Honest miners unknowingly work on invalid chains, reducing their rewards.
 3. **Gain Control Over Blockchain Growth** The attacker tries to influence which chain becomes the main chain.
 4. **Increase Relative Hash Power Advantage** Over time, selfish miners aim to dominate the network by making honest mining less profitable.
-

3. List any two consequences of a 51% attack

A situation where a single entity controls the majority of mining power and can manipulate the blockchain.

1. **Double Spending** The attacker can spend the same cryptocurrency multiple times.
 2. **Transaction Reversal** Previously confirmed transactions can be invalidated, reducing reliability.
-

4. Identify four applications of Smart Contracts

Smart contracts are self-executing digital agreements stored on blockchain platforms like Ethereum, where terms are automatically enforced.

1. **Supply Chain Management** Tracks goods movement and automates payments upon delivery.
2. **Financial Services (DeFi)** Enables lending, borrowing, and trading without intermediaries.

3. **Voting Systems** Ensures transparent and tamper-proof elections.
4. **Insurance Claims Processing** Automatically processes claims when predefined conditions are met.

16 Marks:

1. Describe the concept of Zero Knowledge Proofs and list its key properties

Zero Knowledge Proof is a cryptographic method by which a prover can demonstrate to a verifier that a given statement is true **without revealing any information apart from the validity of the statement itself**. It plays a crucial role in modern privacy-preserving systems such as Zcash.

Concept of Zero Knowledge Proofs (Detailed Explanation)

1. **Fundamental Idea of ZKP:** The core idea behind Zero Knowledge Proof is to separate *verification* from *information disclosure*. In traditional systems, proving something often involves revealing data (e.g., sharing a password). However, in ZKP, the prover proves possession of knowledge without revealing the knowledge itself. This ensures **data confidentiality**, **security**, and **trust** in digital systems.
2. **Mathematical Foundation** ZKPs rely on complex mathematical problems such as:
 - Discrete logarithms
 - Elliptic curve cryptography
 - Hash functionsThese problems are computationally hard to solve, which ensures that even if a proof is intercepted, the secret cannot be deduced.
3. **Participants in ZKP System**
 - **Prover (P):** Holds the secret (witness) and generates the proof.
 - **Verifier (V):** Validates the proof without learning the secret.
 - The interaction is designed so that the verifier gains confidence without gaining knowledge.
4. **Protocol Execution (Step-by-Step)**
 1. The prover starts with a secret input.
 2. The prover generates a commitment (a cryptographic value derived from the secret).
 3. The verifier sends a random challenge.
 4. The prover responds using the secret and challenge.
 5. The verifier checks the response using predefined rules.This process may repeat multiple times to reduce the probability of cheating.
5. **Interactive vs Non-Interactive ZKP**
 - a) **Interactive ZKP**
 - Requires continuous communication between prover and verifier.
 - Example: Challenge-response protocols.
 - More secure but slower due to multiple rounds.
 - b) **Non-Interactive ZKP (NIZK)**
 - Only a single proof is generated and sent.
 - No back-and-forth communication.
 - Widely used in blockchain systems for scalability.
6. **Ali Baba Cave Example (Expanded Explanation)**
 - A circular cave has two paths (A and B) connected by a locked door.
 - The prover claims to know the secret password to open the door.
 - The prover randomly enters one path.
 - The verifier stands outside and asks the prover to come out via a specific path.
 - If the prover knows the password, they can always exit correctly.
 - If not, they can only guess (50% chance).
 - Repeating multiple times reduces cheating probability to near zero.This demonstrates **soundness and zero knowledge**.
7. **Types of Advanced ZKP Systems**

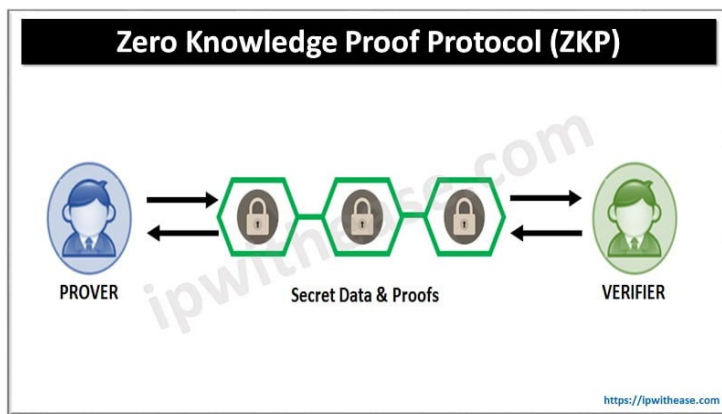
- **zk-SNARKs (Zero-Knowledge Succinct Non-Interactive Arguments of Knowledge):** Very short proofs and fast verification (used in Zcash).
 - **zk-STARKs (Zero-Knowledge Scalable Transparent Arguments of Knowledge):** More scalable and does not require trusted setup.
-

Key Properties of Zero Knowledge Proofs (Expanded)

1. **Completeness** If the statement is true and both prover and verifier follow the protocol correctly, the verifier will always accept the proof. This ensures correctness of the protocol under honest conditions.
 2. **Soundness** If the statement is false, no dishonest prover can convince the verifier except with negligible probability. This protects against fraud and false claims.
 3. **Zero-Knowledge Property** The verifier learns nothing other than the fact that the statement is true. Formally, any information gained during the interaction can be simulated without access to the secret.
 4. **Privacy Preservation** Sensitive data (passwords, keys, personal information) is never revealed during the proof process. This is critical in secure systems like digital identity and financial transactions.
 5. **Non-Transferability** The proof cannot be reused or transferred to convince a third party. This prevents misuse or replay attacks in authentication systems.
 6. **Probabilistic Nature** ZKPs often rely on probability to ensure security. Repeated interactions reduce the chance of a false proof being accepted.
 7. **Efficiency**
Modern ZKP systems aim to minimize:
 - Computational cost
 - Communication overhead
 - Verification time This makes them practical for real-world applications.
 8. **Succinctness**
Proofs are very small in size and can be verified quickly, even for large computations. Important for scalability in distributed systems.
 9. **Trust Assumptions (in some systems):** Some ZKP systems require a trusted setup phase. If compromised, it may affect security (e.g., zk-SNARKs).
-

Applications of Zero Knowledge Proofs (Detailed)

1. **Blockchain and Cryptocurrencies:** ZKPs enable private transactions where sender, receiver, and amount are hidden, as seen in Zcash.
 2. **Authentication Systems:** Users can prove identity without revealing passwords, reducing risk of data breaches.
 3. **Secure Voting Systems:** Ensures voters can prove eligibility and vote validity without revealing their identity or choice.
 4. **Confidential Financial Transactions:** Banks can verify transactions without exposing sensitive financial data.
 5. **Identity and Access Management:** Users can prove attributes (e.g., age > 18) without revealing full identity.
 6. **Cloud Computing Security:** Allows verification of computations performed on remote servers without exposing data.
-



Advantages of Zero Knowledge Proofs

1. **Strong Privacy Preservation:** ZKP ensures that no sensitive information (such as passwords or personal data) is revealed during the verification process, thereby maintaining complete confidentiality.
2. **Enhanced Security:** Since the secret is never transmitted, the risk of interception, hacking, or data leakage is significantly reduced compared to traditional authentication systems.
3. **Authentication Without Disclosure:** Users can prove their identity or credentials without sharing actual data, making ZKP ideal for secure login systems and digital identity verification.
4. **Reduced Risk of Data Breaches:** As no actual sensitive data is stored or transferred, even if the system is attacked, attackers cannot retrieve useful information.
5. **Trustless Verification:** ZKP enables one party to verify another without needing to trust them or rely on intermediaries, which is highly useful in decentralized systems.
6. **Applicability in Blockchain Systems:** ZKPs enable private transactions by hiding transaction details while still ensuring correctness, as implemented in Zcash.
7. **Data Minimization:** Only the necessary proof is shared rather than full data, aligning with modern data protection principles and regulations.
8. **Prevention of Identity Theft:** Since actual credentials are never exposed, the chances of identity theft or impersonation are minimized.

Disadvantages of Zero Knowledge Proofs

1. **High Computational Complexity:** Generating and verifying ZKP proofs requires complex mathematical operations, which may consume significant computational resources.
2. **Implementation Difficulty:** Designing and implementing ZKP systems is technically challenging and requires deep expertise in cryptography.
3. **Performance Overhead:** Compared to traditional methods, ZKPs may introduce additional time delays due to multiple computations and verification steps.
4. **Trusted Setup Requirement (in some systems):** Certain ZKP types (e.g., zk-SNARKs) require an initial trusted setup phase, which, if compromised, can weaken the entire system.
5. **Large Proof Sizes (in some cases):** Some ZKP methods generate large proofs, increasing storage and communication costs (though modern systems reduce this issue).
6. **Limited Awareness and Adoption:** ZKP is still an emerging technology, and lack of widespread understanding limits its adoption in many industries.
7. **Difficult Debugging and Auditing:** Since the system hides information, diagnosing errors or verifying correctness of implementation becomes more complex.
8. **Hardware Requirements:** Advanced ZKP systems may require powerful hardware, especially in large-scale applications like blockchain networks.

-
2. Explain the concept of sharding in blockchain and discuss how sharding-based consensus algorithms improve scalability and performance

3. Explain the various three types of attacks on blockchain networks and discuss their impact on security and trust

Blockchain attacks are deliberate attempts by malicious entities to exploit weaknesses in a blockchain network in order to **manipulate transactions, disrupt consensus, or reduce trust and security** in the system. These attacks target the core principles of blockchain such as decentralization, immutability, and transparency, as seen in networks like Bitcoin.

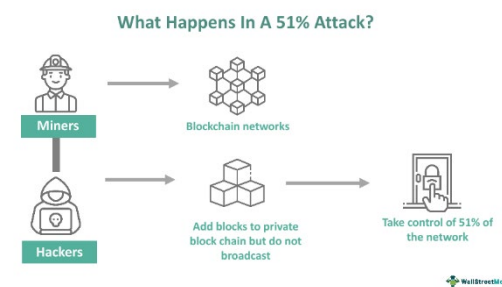
1. 51% Attack (Majority Attack)

A 51% attack occurs when an attacker or group of attackers gains control of more than half of the network's total computational (hash) power, enabling them to control the validation of transactions and blocks.

- Blockchain networks using Proof-of-Work depend on distributed miners.
- Consensus is achieved when the majority agrees on the valid chain.
- If one entity controls >50% of the hash power, they can override honest nodes.
- This allows the attacker to rewrite parts of the blockchain.

Working Mechanism (Step-by-Step)

1. The attacker accumulates majority computational power.
2. Begins mining blocks privately (hidden chain).
3. Sends a transaction on the public blockchain (e.g., payment).
4. Continues mining a private chain where the transaction does not exist.
5. Once the private chain becomes longer, it is released.
6. The network accepts the longer chain, invalidating earlier transactions.



Effects on Security

- Breaks the **immutability** of blockchain.
- Undermines **consensus mechanisms**.
- Enables manipulation of transaction history.

Impact on Trust

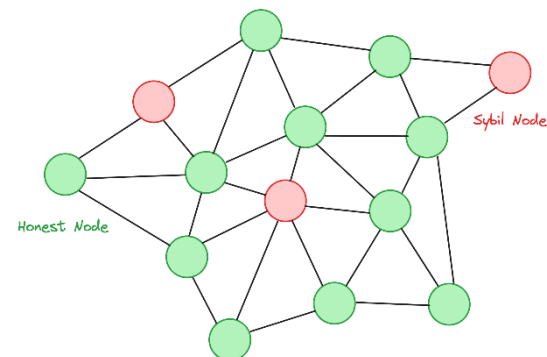
- Users lose confidence in transaction finality.
- Merchants may stop accepting cryptocurrency.
- Investors may withdraw due to insecurity.

Real-World Relevance: Smaller blockchains are more vulnerable due to lower hash power compared to large networks like Bitcoin.

2. Sybil Attack

A Sybil attack occurs when a malicious user creates multiple fake identities or nodes in a blockchain network to gain excessive influence over the system.

- Blockchain networks rely on peer-to-peer communication.
- Each node is assumed to be independent and trustworthy.
- In a Sybil attack, this assumption is violated.
- The attacker controls many nodes, making them appear as multiple independent participants.



Working Mechanism (Step-by-Step)

1. Attacker generates a large number of fake nodes (identities).
2. These nodes join the network and interact with honest nodes.
3. The attacker uses these nodes to influence consensus or voting.
4. The attacker may isolate honest nodes or manipulate data flow.

Effects on Security

- Disrupts **network communication**.
- Weakens **consensus integrity**.
- Increases vulnerability to further attacks like 51% attacks.

Impact on Trust

- Reduces confidence in the fairness of the network.
- Users cannot distinguish between real and fake participants.
- Leads to perceived centralization.

Special Note: Modern blockchains reduce Sybil attacks using mechanisms like Proof-of-Work or Proof-of-Stake, where creating identities is costly.

3. Double Spending Attack

Double spending is a type of attack where a user spends the same cryptocurrency more than once by exploiting flaws in transaction confirmation.

- Digital currencies are just data; they can theoretically be copied.
- Blockchain prevents this using consensus and confirmations.
- Attackers attempt to bypass or delay these mechanisms.

Types of Double Spending Attacks

1. Race Attack

- Two transactions are broadcast simultaneously.
- The attacker hopes the merchant accepts the unconfirmed transaction.

2. Finney Attack

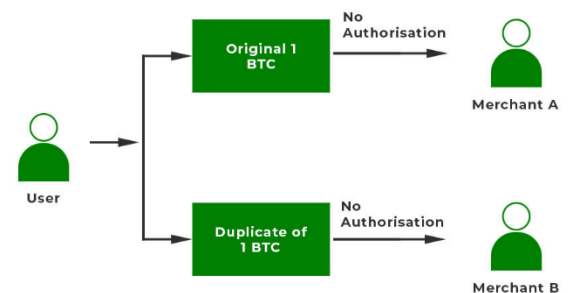
- A miner pre-mines a block containing a transaction to themselves.
- Then makes a payment and later releases the pre-mined block to invalidate it.

3. Vector76 Attack

- Combines race and Finney attacks, targeting lightweight clients.

4. 51% Double Spending

- Uses majority control to reverse confirmed transactions.



Working Mechanism (General Steps)

1. Attacker initiates a payment to a merchant.
2. Simultaneously creates another transaction sending funds back to themselves.
3. Manipulates the network to confirm the second transaction.
4. The original transaction becomes invalid.

Effects on Security

- Breaks **transaction integrity**.
- Undermines **reliability of digital currency**.

Impact on Trust

- Merchants lose money and confidence.
- Users hesitate to accept fast or unconfirmed transactions.
- Slows adoption of blockchain systems.

Comparative Impact of the Three Attacks

Attack Type	Target Area	Main Damage	Trust Impact
51% Attack	Consensus mechanism	Chain manipulation, double spending	Very High
Sybil Attack	Network structure	Fake identities, manipulation	High
Double Spending	Transactions	Financial fraud	Very High

Overall Impact on Blockchain Security and Trust

1. **Compromise of Decentralization:** Attacks centralize control, defeating the purpose of blockchain.

2. **Loss of Immutability:** Transactions can be altered or reversed, weakening reliability.
 3. **Financial Risks:** Users and businesses may suffer direct monetary losses.
 4. **Reputation Damage:** Affected blockchain platforms lose credibility in the market.
 5. **Reduced Adoption:** Fear of attacks discourages new users and investors.
 6. **Increased Operational Costs:** More resources are required to secure networks against attacks.
-

Conclusion (Exam Point)

Attacks such as 51% attacks, Sybil attacks, and double spending highlight critical vulnerabilities in blockchain systems. While blockchain offers strong security features, these attacks demonstrate that maintaining **decentralization, consensus integrity, and user trust** is essential for the long-term success and adoption of blockchain technologies.